

AI Operations (AIOps) — Module 3 Assignment

Infrastructure & Containerization: A Spam-Detection API

Roll No: DA24B025

1 Single-Stage vs. Multi-Stage Docker

Image	Size (docker images)	Reduction
spam-api:naive	2.14GB	—
spam-api:multi	710MB	66.8 %

1.1 Why the multi-stage image is smaller

The naive image is built entirely on `python:3.11`, whose Debian layers carry roughly 980 MB of apt-installed build toolchain — compilers, dev headers, `wget/gpg` — that the running API never touches; the multi-stage build keeps that base in the builder stage only and ships the final image on `python:3.11-slim`. The second saving is everything that never crosses the stage boundary: `generate_dataset.py`, `train_model.py`, the generated `spam_dataset.csv` and the `__pycache__` left by running them all stay in the builder, so the runtime `/app` holds just `model.joblib` (16 KB) and `q1_app.py`. What does *not* shrink is the dependency set — `numpy`, `scipy` and `scikit-learn` are copied across in full, because `q1_app.py` needs them to unpickle the pipeline and call `.predict()`.

2 Multi-Container Orchestration with Docker Compose

2.1 Cache-hit speedup evidence

Request	X-Cache	Latency (ms)	Body
1st (identical text)	MISS	3.068	<code>{"label": "spam"}</code>
2nd (identical text)	HIT	0.152	<code>{"label": "spam"}</code>

A $\approx 20\times$ speedup for an identical answer. The stored entry was confirmed directly in Redis (key `spam:v1:a7339046...`, TTL 247 of the configured 300s), and the server logs show one MISS line followed by one HIT line for the same text.

2.2 Compose vs. Kubernetes

A Dockerfile describes exactly one image, so it cannot express a system made of several containers: Compose is what builds and starts the API and Redis together, puts them on a shared network where each addresses the other by service name, injects per-service configuration, and orders startup behind a healthcheck. Its scope, however, is one Docker daemon, no scheduling decisions, and a container that dies is at best restarted in place. Kubernetes performs the same orchestration across a cluster of nodes, adding a scheduler that places pods against node resources, controllers that recreate lost pods, rolling updates and horizontal scaling; Compose is the right tool for local development and CI, Kubernetes for multi-node production.

3 Kubernetes Indexed Job: Parallel Data Validation

3.1 Parallelism choice for 2 nodes $\times$ 2 CPUs

Validating a 100-row CSV is I/O-bound rather than compute-bound, so each pod requests 500m (requests = limits, so the pods are Guaranteed QoS and the scheduler's arithmetic is exactly the runtime's). Against the stated budget that is nominally $4000m \div 500m = 8$ slots, but the budget is not empty: `kube-system` already requests 750m on the control-plane node and 200m on the worker, leaving 1250m and 1800m respectively — 3050m in total, or 6 slots. Bin-packing reduces that further, because a pod cannot straddle nodes: $\lfloor 1250/500 \rfloor = 2$ on the control plane and $\lfloor 1800/500 \rfloor = 3$ on the worker, for a true ceiling of 5 concurrent pods.

`parallelism: 4` is therefore the chosen value: it sits below that ceiling with ≥ 250 m of slack on each node, so a transient `kube-system` pod cannot strand a shard in `Pending`, and it divides `completions: 8` into exactly two equal waves.

3.2 Evidence the parallelism was reached

NAME	READY	STATUS	NODE
signup-validation-4-nw5tq	1/1	Running	minikube-m02
signup-validation-5-zwzgm	1/1	Running	minikube-m02
signup-validation-6-vhs2n	1/1	Running	minikube
signup-validation-7-zrzld	1/1	Running	minikube-m02

Four pods `Running` simultaneously (the second wave, shards 4–7) — three on the worker and one on the control plane, exactly the 2-and-3 slot ceiling computed above. Each pod holds for 5s before exiting, which is what makes the wave observable rather than a race against `kubect1`. The exact split between nodes varies with whatever else the scheduler has placed; the concurrent count does not.

3.3 Results collected via the Kubernetes API

Shard	0	1	2	3	4	5	6	7
Invalid rows	8	12	6	20	10	17	6	11

90 invalid rows out of 800, matching the seeded generator exactly. A shared `PersistentVolume` is the obvious way to gather these and the wrong one here: minikube's default storage provisioner binds a volume to a single node, so a pod scheduled on the other node cannot reliably read or write it — and the evidence above shows pods landing on both. Each pod therefore prints its count to stdout and `collect_results.py` pulls it back with `read_namespaced_pod_log()`, which behaves identically regardless of where a pod ran.

3.4 Would 3 nodes × 2 CPUs change the choice?

Yes — raise it to 8. A third node adds 1800m of schedulable CPU, lifting the packing ceiling from 5 to $2+3+3=8$, which is exactly `completions`. All eight shards would then run in a single wave instead of two, roughly halving wall-clock time. Any intermediate value gains nothing: at `parallelism: 6` the job still runs in two waves (6 then 2), so the only step that actually changes the wave count is the one that fits all eight at once.

4 Kubernetes Deployments: Self-Healing & Rolling Updates

4.1 Self-healing

```
$ kubectl delete pod spam-api-74c9f66bb5-12bfp
NAME                READY   STATUS    AGE
spam-api-74c9f66bb5-mv4sn  1/1     Running   65s
spam-api-74c9f66bb5-wzccl  0/1     Running   9s
SuccessfulCreate    replicaset/spam-api-74c9f66bb5   Created pod:  spam-api-74c9f66bb5-wzccl
```

The replacement is created by the **ReplicaSet controller** inside `kube-controller-manager` — not by the Deployment directly, which owns the ReplicaSet and delegates pod count to it. It runs a reconciliation loop comparing the number of live pods matching its label selector against `.spec.replicas`; deleting a pod drops the observed count to 1 against a desired 2, and the controller closes the gap by creating one more. The event above names the ReplicaSet as the actor, and the replacement keeps the same `spam-api-74c9f66bb5-` prefix because it comes from the same ReplicaSet.

4.2 Rolling update

`VERSION` was changed from `v1` to `v2` in the API, rebuilt as a new image tag and rolled out with `kubectl set image`. Availability was measured by polling `/healthz` through the Service once a second, 30 times, spanning the rollout.

```
$ kubectl rollout status deployment/spam-api
Waiting ...: 1 out of 2 new replicas have been updated...
Waiting ...: 1 old replicas are pending termination...
deployment "spam-api" successfully rolled out
$ kubectl rollout history deployment/spam-api
REVISION  CHANGE-CAUSE
3         <none>      (v1)
4         <none>      (v2)
probe: requests_ok=30 requests_failed=0 (21 answered v1, 9 answered v2)
```

30 of 30 requests succeeded, and responses alternated between `v1` and `v2` mid-rollout — both versions served simultaneously, which is what `maxSurge: 1` with `maxUnavailable: 0` buys. A Job could not have done this at all: a Job's `.spec.template` is immutable once created, so there is no in-place image change to make — the Job would have to be deleted and recreated, its pods would run once and exit rather than stay up behind the Service, and there would be no revision history to `rollout undo` to.

4.3 Deployment vs. Job

A Deployment treats a pod's exit as a failure to be corrected, which is right for a server whose whole purpose is to stay reachable: it maintains a replica count indefinitely, replaces anything that dies, and can swap image versions gradually because there are always healthy replicas to absorb traffic. A Job treats a pod's exit with status 0 as success and stops, which is right for the shard validation in Q3 — that work has a defined end, and a controller that kept restarting completed pods would loop forever. The distinction is whether the workload has a terminating condition: serving does not, batch does.